

Architecture de Scoring Binaire pour les Prêt à la Consommation

Rafael CEREZO MARTIN

Février 2026

Objectifs du projet

Département demandeur : Direction des Risques Financiers

Problématique : Inclusion financière des clients sans historique de crédit tout en minimisant le risque de défaut.

Objectif principal : Développer un modèle de classification automatisé pour prédire la probabilité de faillite et industrialiser son cycle de vie via une approche MLOps de bout en bout.

Standard Industriel (MLOps)

- **Tracking avec MLflow :** Suivi rigoureux des expérimentations, des hyperparamètres et des métriques métiers pour garantir la reproductibilité totale.
- **Model Registry & Serving** : Stockage centralisé et versionnage des modèles pour faciliter le passage de l'expérimentation au test de serving.
- **Optimisation Avancée :** Utilisation d'Optuna ou GridSearchCV pour maximiser les performances techniques et métiers.

Exigences Métiers & Fiabilité

- **Score "Métier" Personnalisé** : Optimisation du seuil de classification pour minimiser le coût d'erreur (coût d'un Faux Négatif estimé 10x supérieur à celui d'un Faux Positif).
- **Transparence & Interprétabilité** : Analyse de la Feature Importance globale et locale pour expliquer chaque décision de crédit de manière compréhensible.
- **Robustesse des Données :** Traitement avancé du déséquilibre des classes (SMOTE ou class_weight) et validation croisée (StratifiedKFold).

i Conception Préliminaire

1 Analyse Exploratoire (Nettoyage et Préparation)

2 Feature Engineering (Preprocessing)

3 Enregistrer Expérimentations dans MLFlow

4 Optimisation hyperparamètres

Architecture de Scoring Binaire pour les Prêt à la Consommation

Initialisation du projet

1. Création de la structure

2. Configuration des libraires

Unification de l'infrastructure via Docker Compose

3. PostgreSQL: Persistance et Gouvernance des Données

4. pgAdmin: Administration de la base de données via

5. MLflow: Orchestration du Cycle de Vie

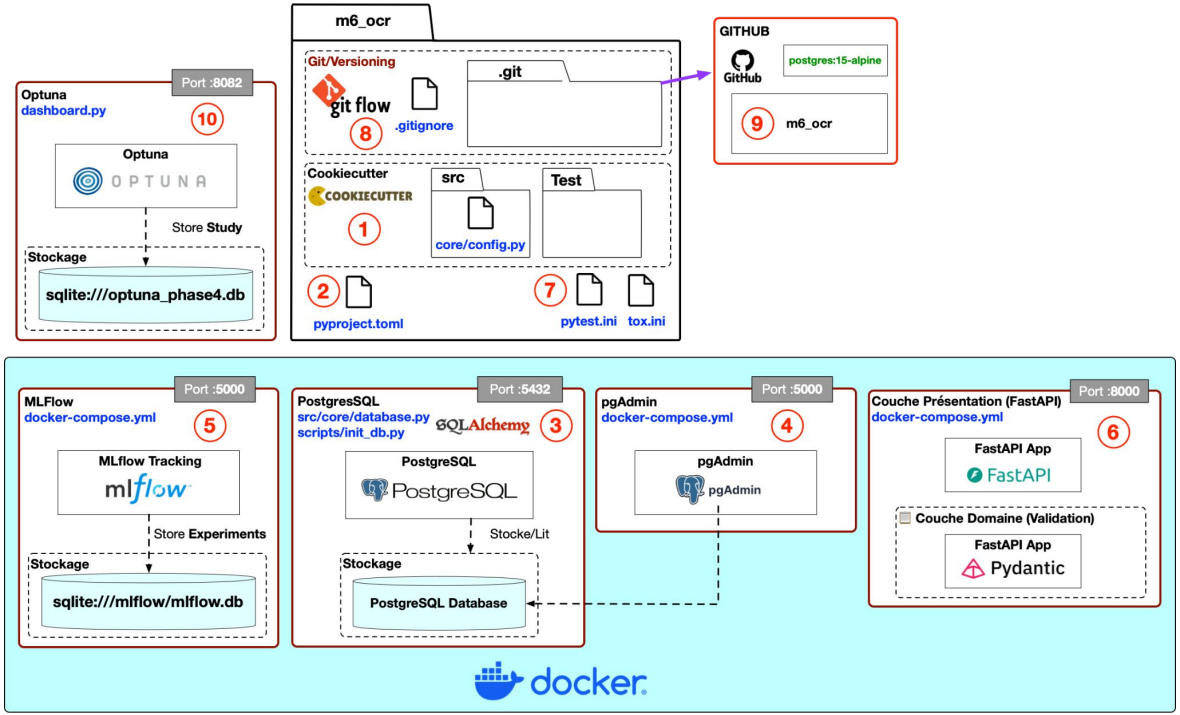
6. FastAPI: Déploiement et Consommation

7. Test initiaux de vérification des services.

8. Initialization du repository local de Versioning GIT

9. Création du repository distant sur GitHub.

10. Optuna: Monitorer l'optimisation des hyperparamètres



Contexte Métier

1. Création de la structure

2. Configuration des libraires

Unification de l'infrastructure via Docker Compose

3. PostgreSQL: Persistance et Gouvernance des Données

4. pgAdmin: Administration de la base de données via

5. MLflow: Orchestration du Cycle de Vie

6. FastAPI: Déploiement et Consommation

7. Test initiaux de vérification des services.

8. Initialization du repository local de Versioning GIT

9. Création du repository distant sur GitHub.

10. Optuna: Monitorer l'optimisation des hyperparamètres

Contexte Métier & Critères d'Optimisation

Cible (TARGET)
0 = Bon payeur | 1 = Défaut de paiement

Population
Clients sans historique bancaire — données hétérogènes

Defi
Déséquilibre fort des classes (~92% vs 8%)

Source données
Kaggle — Home Credit + tables annexes fusionnées

⚠️ Asymétrie des Coûts d'Erreur

Faux Négatif (FN)
Mauvais client ⟶ crédit accordé
Perte en capital
COÛT × 10

Faux Positif (FP)
Bon client ⟶ crédit refusé
Manque à gagner en marge
COÛT × 1

📊 Métriques & Critère d'Optimisation

Score Métier (principal)	min(10×FN + 1×FP) — seuil optimisé
AUC-ROC	Mesure de discrimination globale
Accuracy / F1	Contrôle et comparaison secondaire

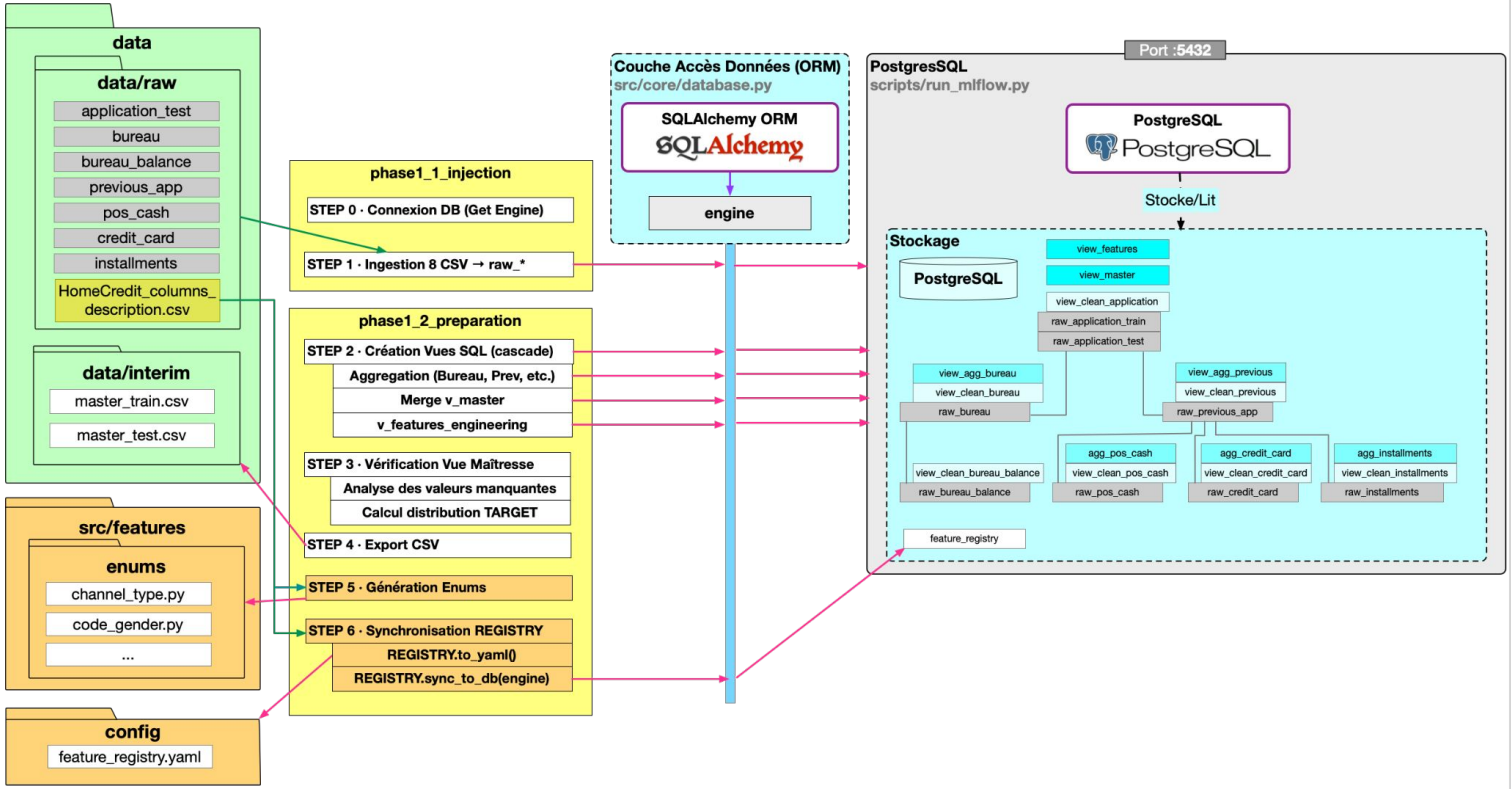
1

Analyse Exploratoire (Nettoyage et Préparation)

- STEP 0 · Connexion DB (Get Engine)
- STEP 1 · Ingestion 8 CSV → raw_*
- STEP 2 · Création Vues SQL (cascade)
- STEP 3 · Vérification Vue Maîtresse
- STEP 4 · Export CSV
- STEP 5 · Génération Enums
- STEP 6 · Synchronisation REGISTRY

Analyse Exploratoire (Nettoyage et Préparation)

STEP 1. Pipeline



2

Feature Engineering (Preprocessing)

STEP 1. Chargement des données

STEP 2. Validation du DataFrame contre le REGISTRY

STEP 3. Fit sur le train

STEP 4. Target encoding (colonnes haute cardinalité)

STEP 5. Transform train

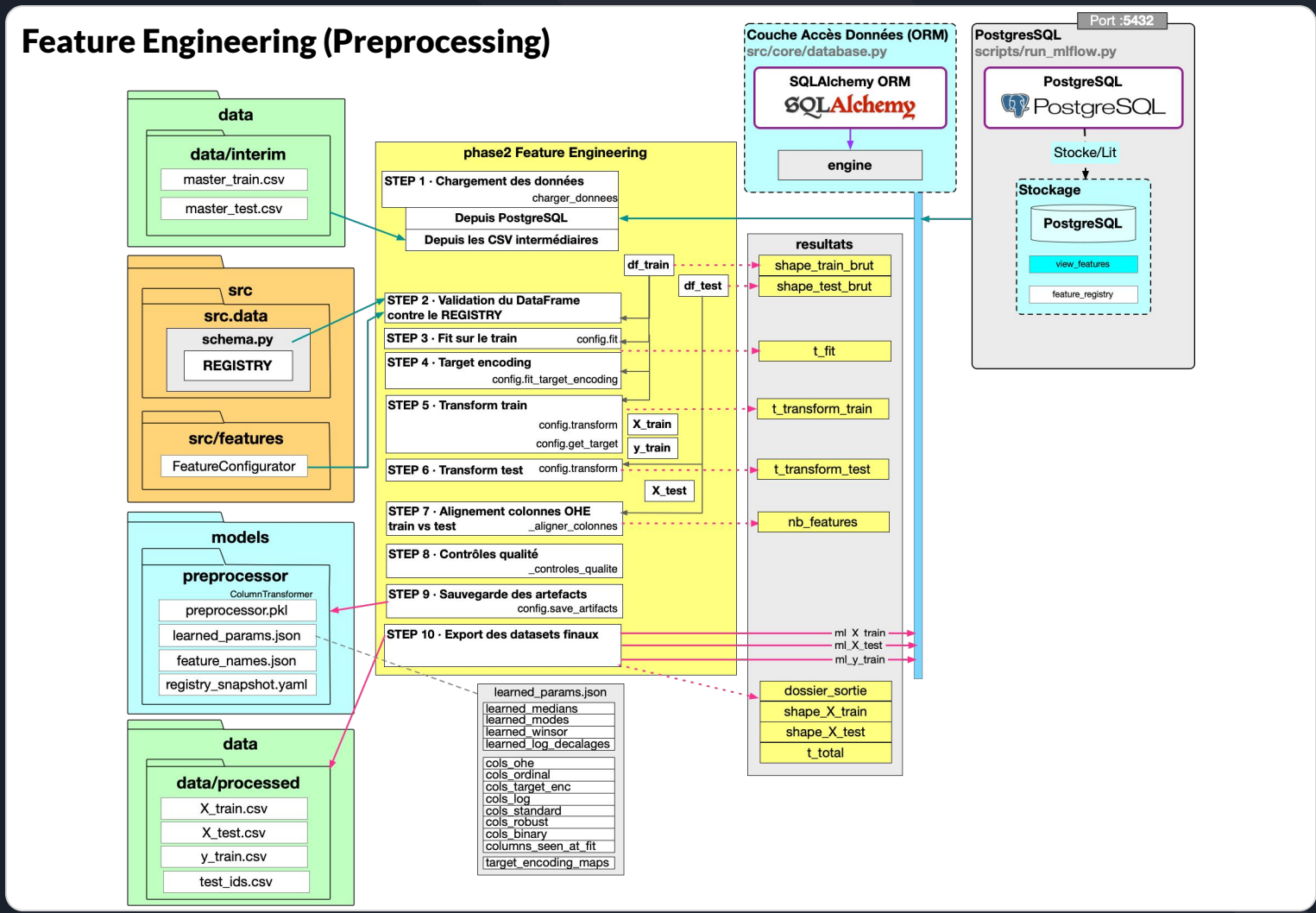
STEP 6. Transform test (mêmes paramètres que le train)

STEP 7. Alignement colonnes OHE train vs test

STEP 8. Contrôles qualité

STEP 9. Sauvegarde des artefacts sklearn

STEP 10. Export des datasets finaux



3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

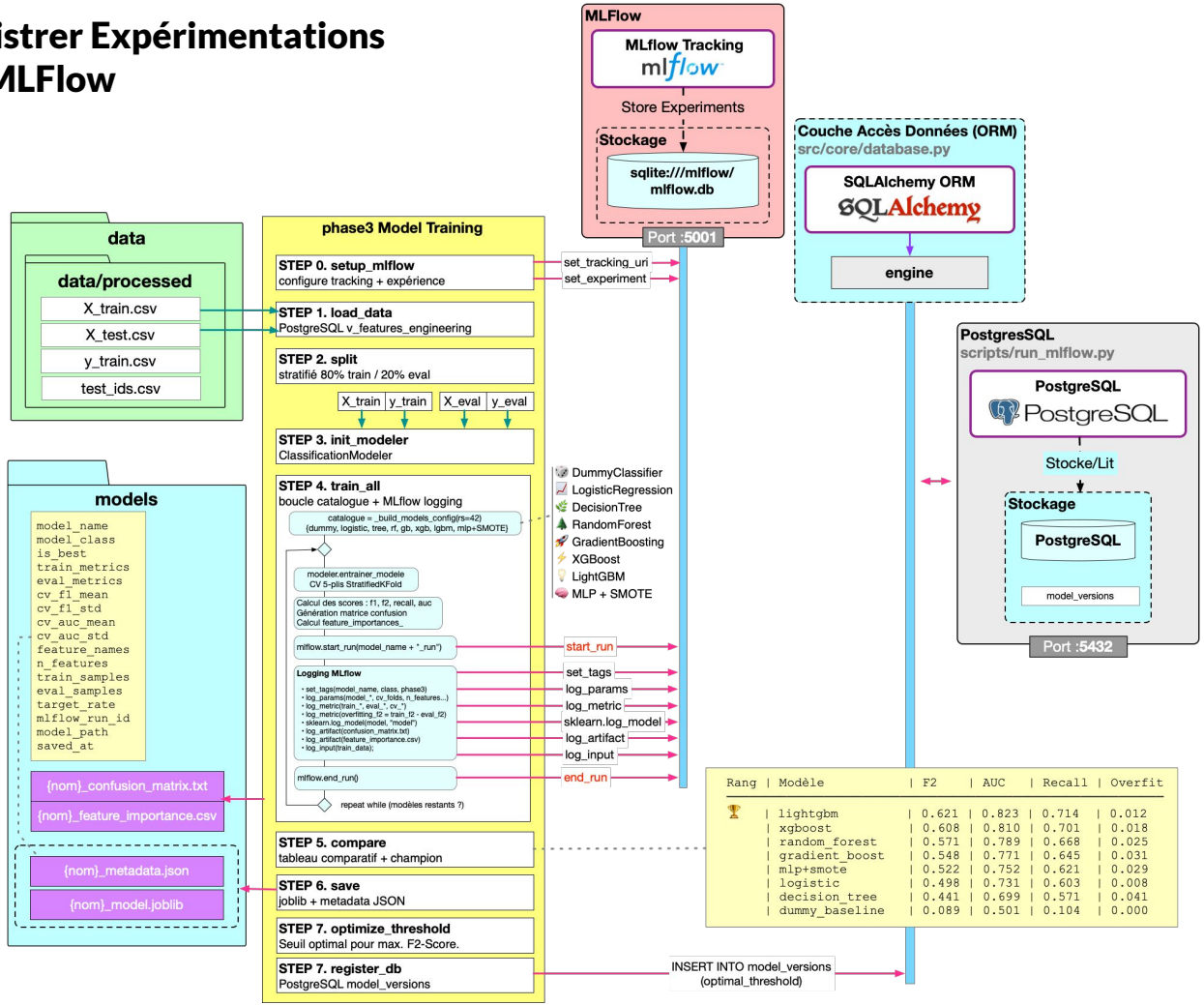
STEP 4. train_all:
boucle catalogue + MLflow logging

STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

STEP 7. register_db:
PostgreSQL model_versions

Enregistrer Expérimentations dans MLFlow



4

Optimisation hyperparamètres

STEP 0. setup_mlflow:
configure + récupérer expérience

STEP 1. load_champion:
Charge modèle champion (joblib + metadata JSON)

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. GridSearchCV:
StratifiedKFold 5-plis + scorer F2

STEP 4. Optuna:
Optimisation bayésienne

STEP 5. compare_engines:
(GridSearch vs Optuna) selon F2 eval

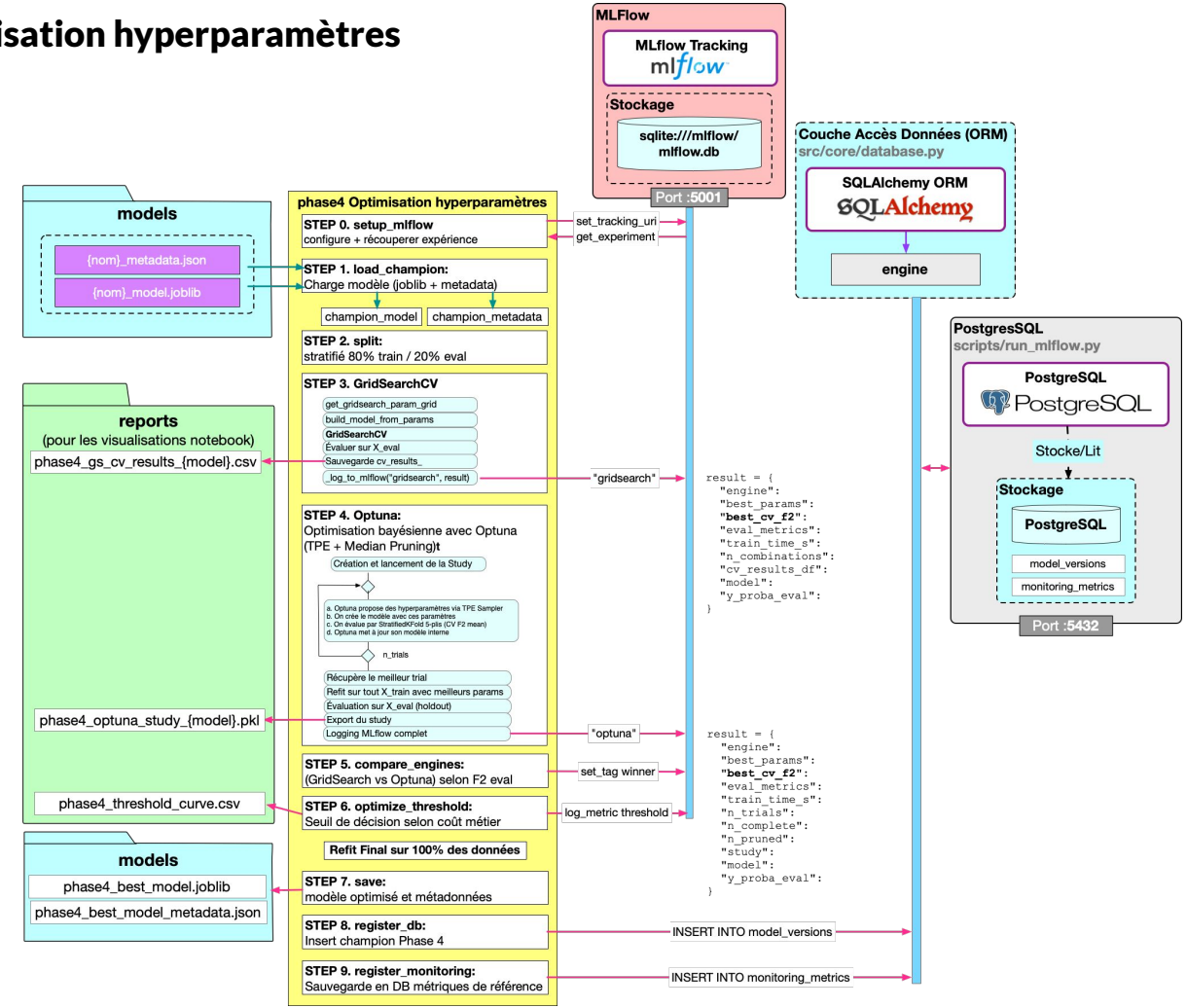
STEP 6. optimize_threshold:
seuil de décision optimal selon coût métier

STEP 7. save:
modèle optimisé et métadonnées

STEP 8. register_db:
Insert champion Phase 4

STEP 9. register_monitoring:
Sauvegarde en DB métriques de référence

Optimisation hyperparamètres



1

Analyse Exploratoire

(Nettoyage et Préparation)

STEP 0 · Connexion DB (Get Engine)

STEP 1 · Ingestion 8 CSV → raw_*

STEP 2 · Création Vues SQL (cascade)

STEP 3 · Vérification Vue Maîtresse

STEP 4 · Export CSV

STEP 5 · Génération Enums

STEP 6 · Synchronisation REGISTRY

1. Analyse Exploratoire (Nettoyage et Préparation)

✅ Exploré les données brutes, identifié les formats et nettoyé les données (valeurs manquantes, doublons, jointures)

Fichier	Description	Granularité	Lignes	Colonnes	Cols avec Nuls	% Cellules Nulles	Doublons	Mémoire (Mo)
application_train	Demande de crédit principale (Entraînement)	1 ligne / client	307 511	122	67	24,4%	0	529,5
application_test	Demande de crédit principale (Test)	1 ligne / client	48 744	121	64	23,8%	0	83,6
bureau	Historique des crédits externes (Bureau de crédit)	N lignes / client	1 716 428	17	7	13,5%	0	495,8
bureau_balance	État mensuel des crédits du Bureau	N lignes / crédit	27 299 925	3	0	0,0%	0	1 801,8
previous_application	Demandes précédentes chez Home Credit	N lignes / client	1 670 214	37	16	18,0%	0	1 785,7
POS_CASH_balance	Soldes mensuels points de vente et cash	N lignes / contrat	10 001 358	8	2	0,1%	0	1 112,5
credit_card_balance	Soldes mensuels de cartes de crédit	N lignes / contrat	3 840 312	23	9	6,7%	0	887,5
installments_payments	Historique des paiements d'échéances	N lignes / paiement	13 605 401	8	2	0,0%	0	870,7

✅ Construit des features pertinentes, y compris via des agrégats et transformations.

```
CREATE OR REPLACE VIEW v_agg_pos_cash AS
SELECT
    sk_id_curr,
    COUNT(*)                          AS pos_records_count,
    AVG(months_balance)               AS pos_months_balance_mean,

    MAX(sk_dpd)                       AS pos_sk_dpd_max,
    AVG(sk_dpd)                       AS pos_sk_dpd_mean,

    MAX(sk_dpd_def)                   AS pos_sk_dpd_def_max,
    AVG(cnt_instalment)               AS pos_cnt_instalment_mean,
    AVG(cnt_instalment_future)        AS pos_cnt_instalment_future_mean,
    SUM(CASE WHEN sk_dpd > 0 THEN 1 ELSE 0 END) AS pos_dpd_count
FROM v_clean_pos_cash
GROUP BY sk_id_curr
```

1

Analyse Exploratoire

(Nettoyage et Préparation)

STEP 0 · Connexion DB (Get Engine)

STEP 1 · Ingestion 8 CSV → raw_*

STEP 2 · Création Vues SQL (cascade)

STEP 3 · Vérification Vue Maîtresse

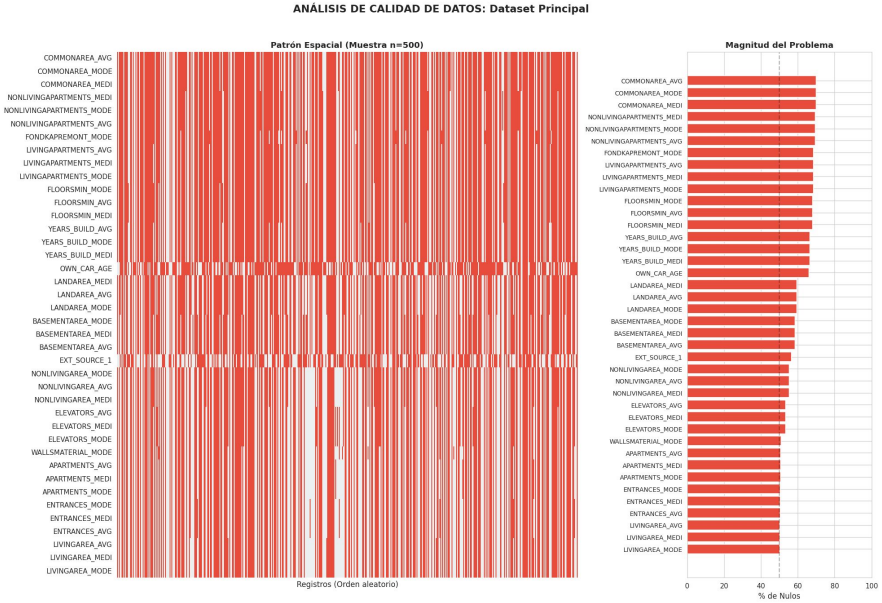
STEP 4 · Export CSV

STEP 5 · Génération Enums

STEP 6 · Synchronisation REGISTRY

1. Analyse Exploratoire (Nettoyage et Préparation)

✅Visualiser les valeurs manquantes: Missing Values (Custom Function)




```
# ── 4. Standard (Version avec Indicateur de Valeurs Manquantes) ──────
self._log("\n    ── 4. Standard (Imputation Médiane + Indicateur) ...")

if self.cols_standard_active:
    std_pipe = Pipeline([
        # add_indicator=True : Crée une colonne binaire supplémentaire
        # pour chaque feature où une valeur manquante a été détectée.
        ("imputer", SimpleImputer(strategy="median", add_indicator=True)),
        ("scaler",  StandardScaler())
    ])
    transformers.append(("standard", std_pipe, self.cols_standard_active))
```

1

Analyse Exploratoire

(Nettoyage et Préparation)

STEP 0 · Connexion DB (Get Engine)

STEP 1 · Ingestion 8 CSV → raw_*

STEP 2 · Création Vues SQL (cascade)

STEP 3 · Vérification Vue Maîtresse

STEP 4 · Export CSV

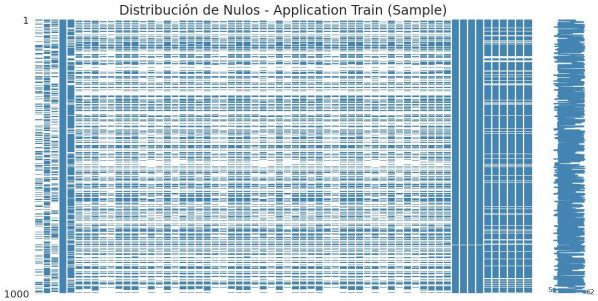
STEP 5 · Génération Enums

STEP 6 · Synchronisation REGISTRY

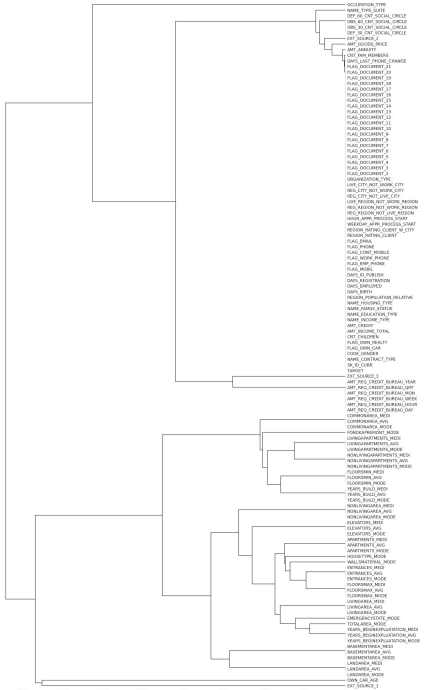
1. Analyse Exploratoire (Nettoyage et Préparation)

✅Visualiser les valeurs manquantes: Missingno

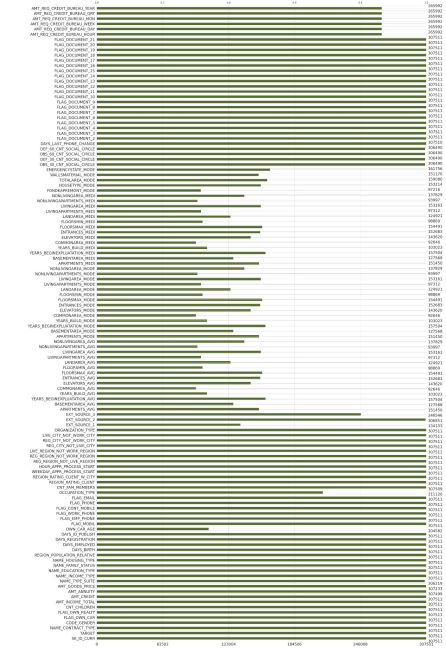
msno.matrix.



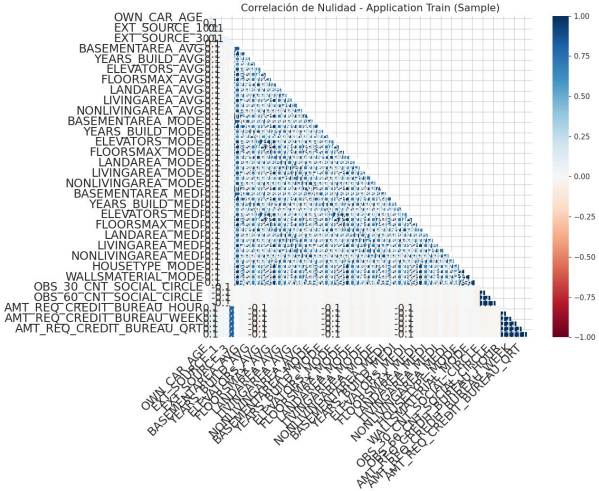
msno.dendrogram



msno.bar



msno.heatmap



1

Analyse Exploratoire

(Nettoyage et Préparation)

STEP 0 · Connexion DB (Get Engine)

STEP 1 · Ingestion 8 CSV → raw_*

STEP 2 · Création Vues SQL (cascade)

STEP 3 · Vérification Vue Maîtresse

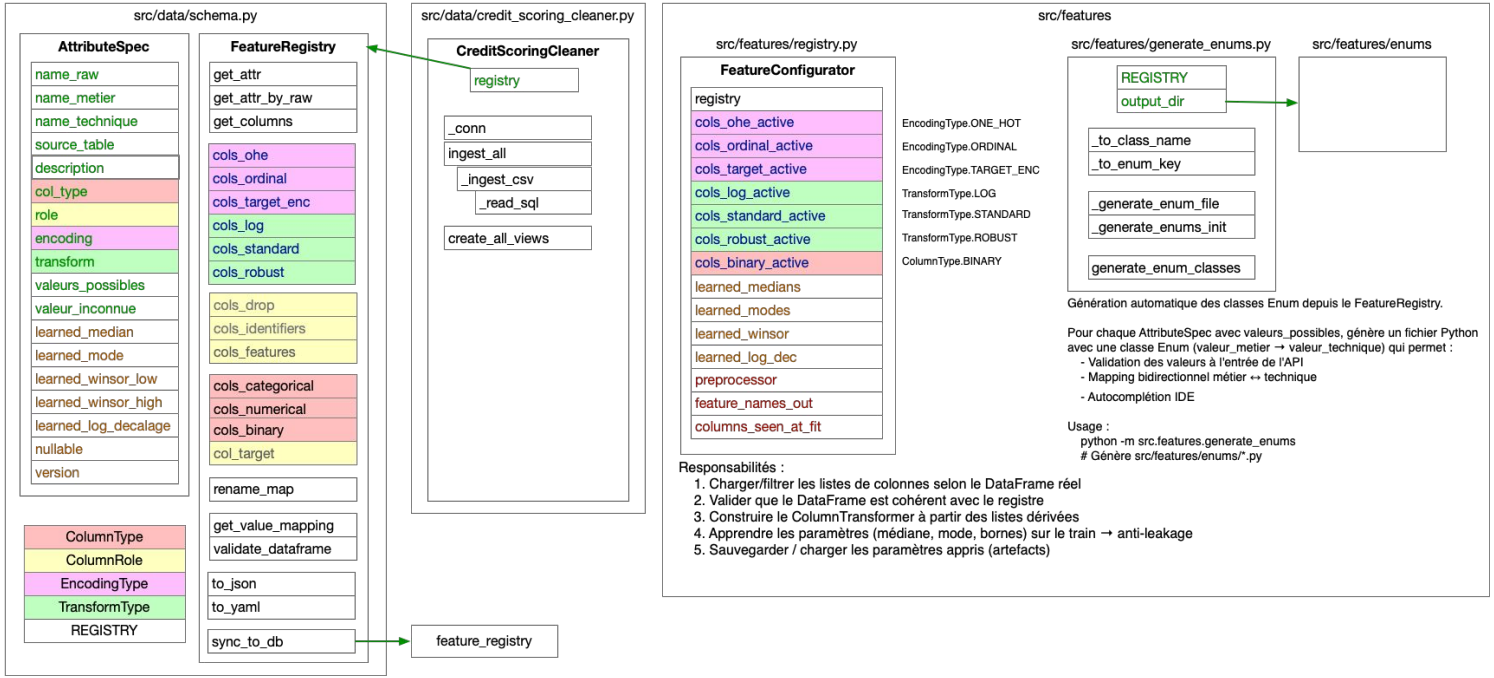
STEP 4 · Export CSV

STEP 5 · Génération Enums

STEP 6 · Synchronisation REGISTRY

1. Analyse Exploratoire (Nettoyage et Préparation)

Génération automatique des **classes Enum** depuis le FeatureRegistry
pour le **Mapping bidirectionnel** métier ↔ technique



2

Feature Engineering (Preprocessing)

STEP 1. Chargement des données

STEP 2. Validation du DataFrame contre le REGISTRY

STEP 3. Fit sur le train

STEP 4. Target encoding (colonnes haute cardinalité)

STEP 5. Transform train

STEP 6. Transform test (mêmes paramètres que le train)

STEP 7. Alignement colonnes OHE train vs test

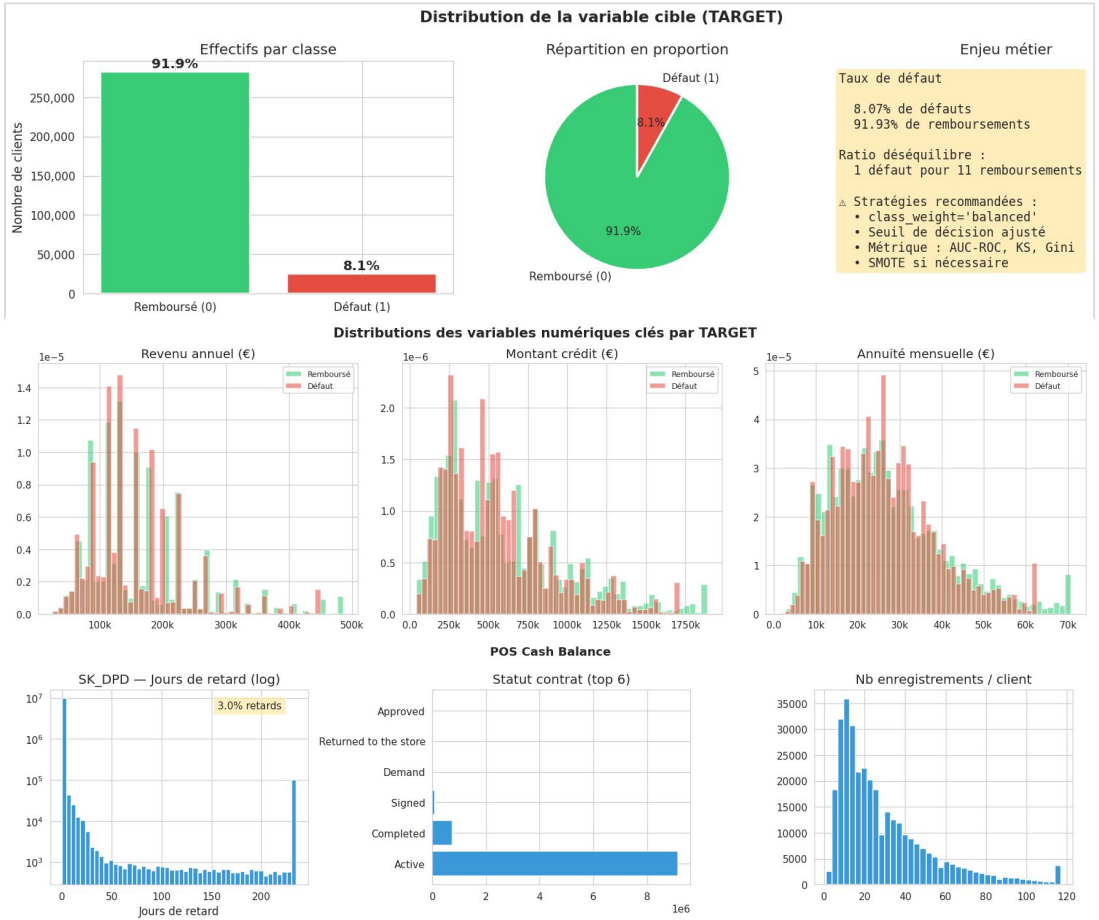
STEP 8. Contrôles qualité

STEP 9. Sauvegarde des artefacts sklearn

STEP 10. Export des datasets finaux

2. Feature Engineering (Preprocessing)

✅ **Valider la qualité des features avec visualisations ou stats descriptives.**



2

Feature Engineering (Preprocessing)

STEP 1. Chargement des données

STEP 2. Validation du DataFrame contre le REGISTRY

STEP 3. Fit sur le train

STEP 4. Target encoding (colonnes haute cardinalité)

STEP 5. Transform train

STEP 6. Transform test (mêmes paramètres que le train)

STEP 7. Alignement colonnes OHE train vs test

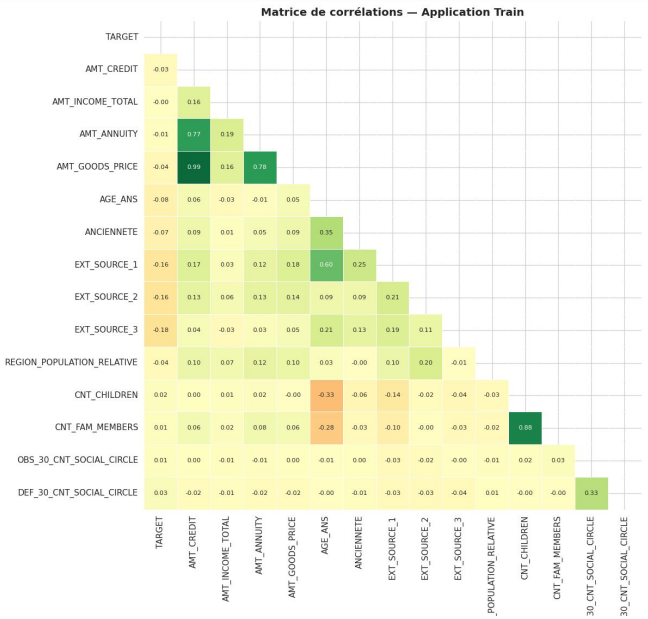
STEP 8. Contrôles qualité

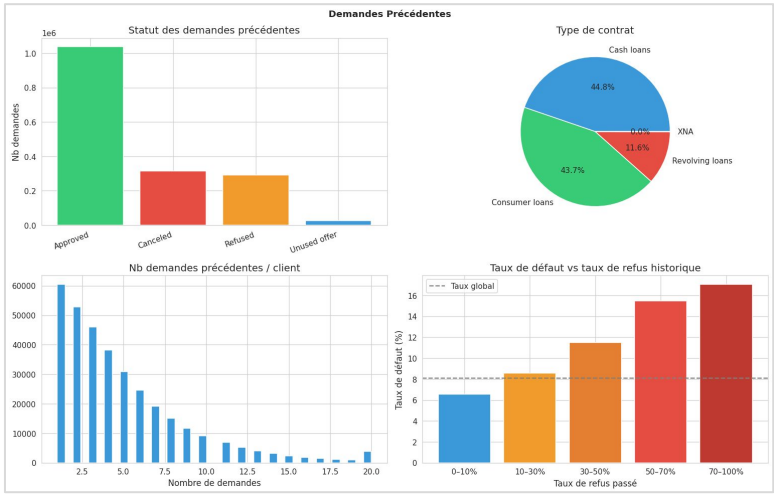
STEP 9. Sauvegarde des artefacts sklearn

STEP 10. Export des datasets finaux

2. Feature Engineering (Preprocessing)

✅ **Valider la qualité des features avec visualisations ou stats descriptives.**





3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

STEP 4. train_all:
boucle catalogue + MLflow logging

STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

STEP 7. register_db:
PostgreSQL model_versions

3. Enregistrer Expérimentations dans MLFlow

✅ **Testé plusieurs modèles de classification.**

#	Modèle	Hyperparamètres clés	Stratégie déséquilibre	Complexité	Rôle
🎲	**Dummy Baseline** DummyClassifier	strategy = stratified random_state = 42	Distribution aléatoire respectant les 8%	NULLE	Plancher de référence — tout modèle réel doit le battre
📈	**Logistic Regression** LogisticRegression	C = 0.1 solver = lbfgs max_iter = 1000	class_weight = **balanced**	BASSE	Interprétable, coefficients lisibles, baseline linéaire
🌿	**Decision Tree** DecisionTreeClassifier	max_depth = 5 min_samples_leaf = 50	class_weight = **balanced**	BASSE	Diagnostic des interactions simples, arbre lisible
🌲	**Random Forest** RandomForestClassifier	n_estimators = 150 max_depth = 10 n_jobs = -1	class_weight = **balanced_subsample**	MOYENNE	Bagging, robuste aux outliers, réduit la variance
🚀	**Gradient Boosting** GradientBoostingClassifier	n_estimators = 100 learning_rate = 0.05 max_depth = 4	⚠️ Sans class_weight natif Poids via sample_weight	MOYENNE	Boosting séquentiel sklearn, stable et robuste
⚡	**XGBoost** XGBClassifier	n_estimators = 200 max_depth = 4 learning_rate = 0.05 n_jobs = -1	scale_pos_weight = **11.5** ≈ ratio neg/pos (1:11)	HAUTE	Performance optimisée, calibré pour le déséquilibre fort
💡	**LightGBM** LGBMClassifier	n_estimators = 200 max_depth = 5 learning_rate = 0.05 n_jobs = -1	class_weight = **balanced** Force Recall > 0	HAUTE	Très rapide sur 300k lignes, Recall prioritaire → candidat champion
🧠	**MLP + SMOTE** MLPClassifier + Pipeline	hidden_layers = (64, 32) alpha = 0.01 max_iter = 300 early_stopping = True	SMOTE (suréchantillonnage synthétique) imblearn Pipeline	HAUTE	Relations non-linéaires complexes, données synthétiques pour équilibrer

3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

STEP 4. train_all:
boucle catalogue + MLflow logging

STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

STEP 7. register_db:
PostgreSQL model_versions

3. Enregistrer Expérimentations dans MLFlow

✅ MLflow — Journal d'Expériences & Model Registry

mlflow.start_run()

Chaque experience tracee avec un nom unique
Organisation en experiments par algorithme

🏷️ **3. Tags (Identification & Audit)**

- **model_name** : Nom métier (ex: XGBoost_V1).
- **model_class** : Nom de la classe Python (ex: XGBClassifier).
- **debug_mode** : Booléen (True/False) pour isoler les tests.
- phase : Étape du cycle de vie (phase3).

Analyse de Performance Multi-Niveaux (Metrics)

On ne logges pas un score, mais une hiérarchie de scores :

- **Performance Brute :** Comparaison systématique train_ vs eval_ (Holdout).
- **Stabilité (Validation Croisée) :** Moyenne ET écart-type (mean & std) pour détecter les modèles instables.
- **Indicateurs d'Overfitting :** Calcul personnalisé des écarts (overfitting_f1, overfitting_f2) pour identifier instantanément si le modèle a "appris par cœur".
- **Efficacité :** train_time_s pour surveiller le coût computationnel

Params & Metriques logges

🔹 **1. Paramètres (Configuration du Run)**

- **model_params** : Hyperparamètres spécifiques (ex: n_estimators, max_depth, learning_rate).
- **random_state** : Graine d'aléatoire pour la reproductibilité.
- **cv_folds** : Nombre de plis (K-folds) utilisés (Fixé à 5).
- **eval_ratio** : Proportion du jeu de données réservée au test (Holdout).
- **n_features** : Nombre total de variables d'entrée (52 colonnes après FE).
- **train_samples** / eval_samples : Taille exacte des sets d'entraînement et de validation.
- **target_rate** : Taux de défaut (moyenne de y_train) pour surveiller le déséquilibre.

📈 **2. Métriques (Performance & Stabilité)**

- **train_[metric]** : Score sur le set d'entraînement (Fit complet).
- **eval_[metric] :** Score sur le set de test (Généralisation).
- **Metrics incluses** : F1, F2, Precision, Recall, AUC.
- **cv_[metric]_mean / std** : Stabilité moyenne et écart-type sur les 5 folds.
- **cv_train_[metric]_mean / std** : Performance moyenne sur les folds d'entraînement.
- **overfitting_f1 / f2** : Différence absolue entre Train et Eval (Indicateur de risque).
- **train_time_s** : Durée de l'entraînement en secondes.

Introspection & Gouvernance (Artifacts & Registry)

On ne logges pas un score, mais une hiérarchie de scores :

- **Signature du Modèle :** Utilisation de infer_signature pour figer le schéma I/O (Input/Output). Cela empêche une API de crasher en production si les colonnes changent.
- **Explicabilité** : Sauvegarde automatique de la Feature Importance (format CSV) et de la Matrice de Confusion (format TXT) pour une analyse post-mortem rapide sans rouvrir un notebook.
- **Traçabilité des Données** : Emploi de mlflow.data pour lier le run à la vue PostgreSQL source (v_features_engineering), assurant une reproductibilité totale.

3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

STEP 4. train_all:
boucle catalogue + MLflow logging

STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

STEP 7. register_db:
PostgreSQL model_versions

3. Enregistrer Expérimentations dans MLFlow

✅ **Utilisé la validation croisée pour comparer les modèles.**

```
# StratifiedKFold pour maintenir la proportion des classes
# ------------------------------------------------------------------------
cv              =StratifiedKFold(
    n_splits     = self.config['CV_FOLDS'],
    shuffle      = True,
    random_state = self.config['RANDOM_STATE']
)                                           # Configuration du split

resultats_cv    = cross_validate(
    estimator          = modele,
    X                  = self.X_train,
    y                  = self.y_train,
    cv                 = cv,
    scoring            = self.config['METRICS'],
    return_train_score = True,
    n_jobs             = -1                 # Utilisation multi-cœurs
)                                           # Calcul des scores
```

✅ **Pris en compte le déséquilibre des classes.**
✅ **Intégré un traitement du déséquilibre (pondération, SMOTE...)**

L'étape du "Target Encoding"
Remplace une catégorie par la moyenne de la cible (le taux de défaut).

L'étape de l'Échantillonnage
Techniques comme :
- le SMOTE (sur-échantillonnage de la classe minoritaire) ou
- le Under-sampling (réduction de la classe majoritaire).

L'étape de l'Entraînement
Parametres des modeles
- is_unbalance=True
- scale_pos_weight

3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

STEP 4. train_all:
boucle catalogue + MLflow logging

STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

STEP 7. register_db:
PostgreSQL model_versions

3. Enregistrer Expérimentations dans MLFlow

✅ **Choisi des métriques adaptées au contexte métier.**

```
DEFAULT_METRICS = {
    'accuracy'  : 'accuracy',                    # Justesse
    'precision' : make_scorer(precision_score,
                              average='binary',
                              zero_division=0), # Précision binaire
    'recall'    : make_scorer(recall_score,
                              average='binary',
                              zero_division=0), # Rappel binaire
    'f1'        : make_scorer(f1_score,
                              average='binary',
                              zero_division=0), # F1-Score

    # ⚠️ MÉTRIQUE PRIORITAIRE : F-beta avec β=2
    # β=2 pénalise 2× plus les faux négatifs que les faux positifs
    'f2'       : make_scorer(fbeta_score     , average='binary', zero_division=0, beta=2  ),

    'roc_auc'   : 'roc_auc',                     # AUC-ROC
    'log_loss'  : 'neg_log_loss'                 # Log-Loss négative
}
```

3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

STEP 4. train_all:
boucle catalogue + MLflow logging

STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

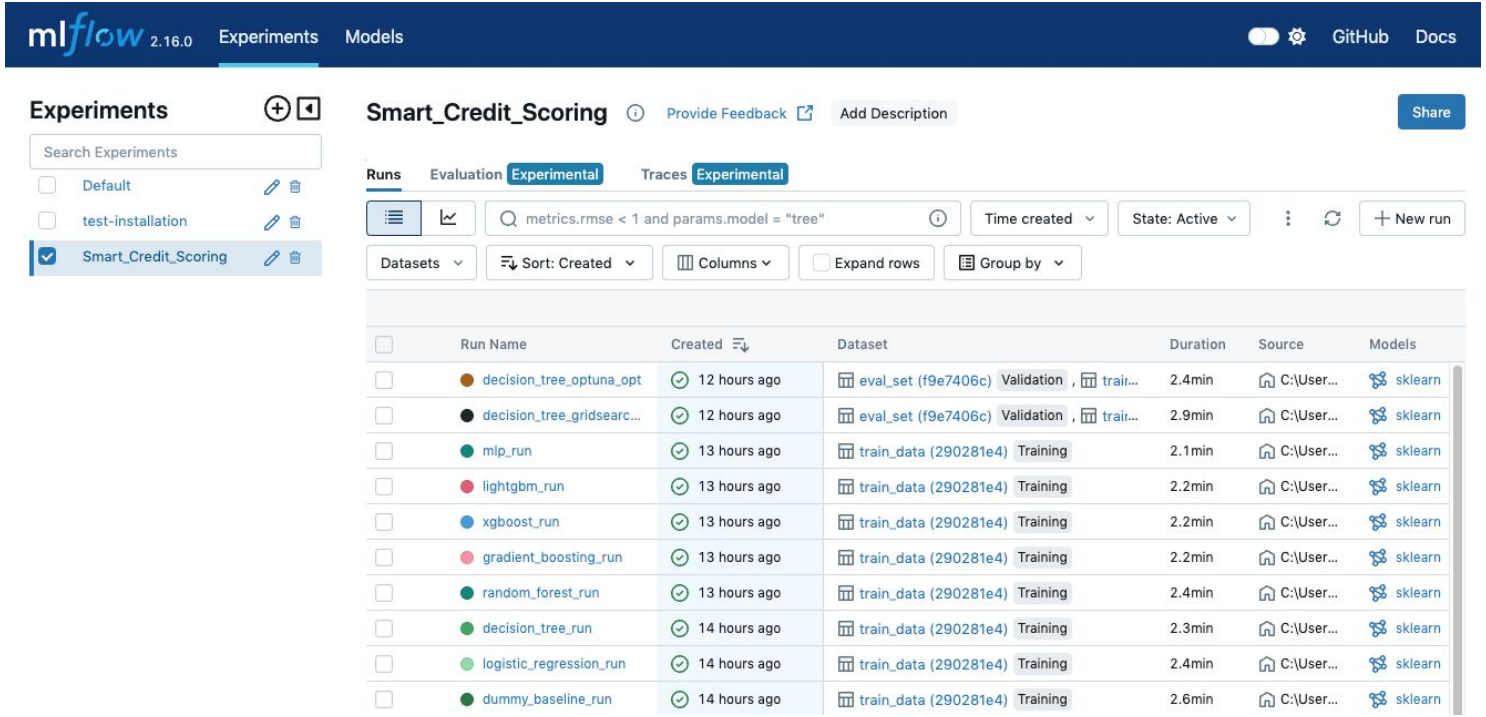
STEP 7. register_db:
PostgreSQL model_versions

3. Enregistrer Expérimentations dans MLFlow

✅ **Utilisé MLflow pour tracker les paramètres, métriques et modèles.**

✅ **Screenshot de l'interface MLflow pour illustrer mes runs**

✅ **Lancé l'interface MLflow ui pour visualiser mes résultats.**



3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

STEP 4. train_all:
boucle catalogue + MLflow logging

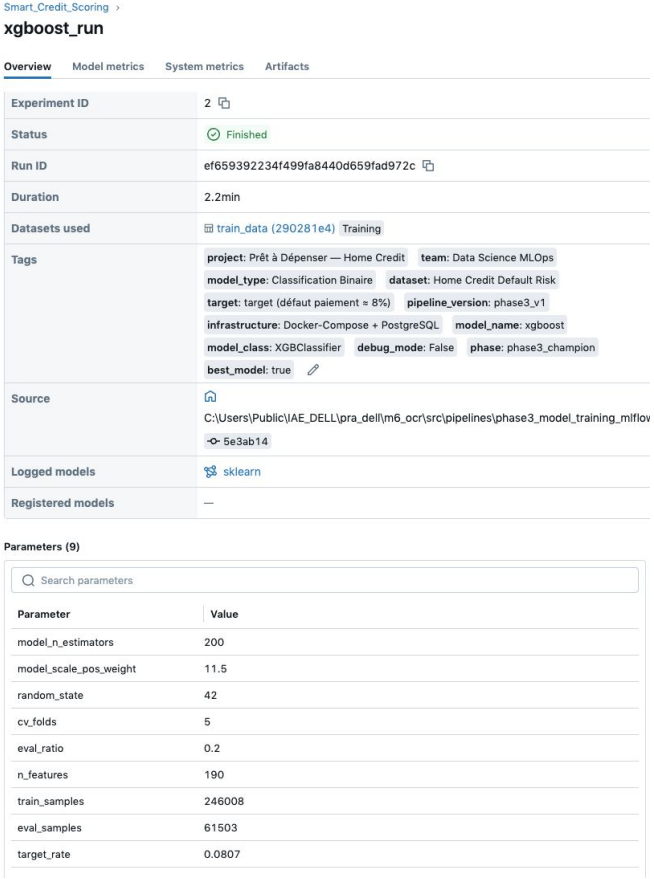
STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

STEP 7. register_db:
PostgreSQL model_versions

3. Enregistrer Expérimentations dans MLFlow

✅ **Annotation des expériences de façon claire (nom, tag, commentaires)**
✅ **Enregistrement des modèles dans un model registry**



Smart_Credit_Scoring ›

xgboost_run

Overview | Model metrics | System metrics | Artifacts

Experiment ID	2
Status	Finished
Run ID	ef659392234f499fa8440d659fad972c
Duration	2.2min
Datasets used	train_data (290281e4) Training
Tags	project: Prêt à Dépenser — Home Credit; team: Data Science MLOps; model_type: Classification Binaire; dataset: Home Credit Default Risk; target: target (défaut paiement ≈ 8%); pipeline_version: phase3_v1; infrastructure: Docker-Compose + PostgreSQL; model_name: xgboost; model_class: XGBClassifier; debug_mode: False; phase: phase3_champion; best_model: true
Source	C:\Users\Public\IAE_DELL\pra_dell\m6_ocr\src\pipelines\phase3_model_training_mlflow; 5e3ab14
Logged models	sklearn
Registered models	—

Parameters (9)

Search parameters

Parameter	Value
model_n_estimators	200
model_scale_pos_weight	11.5
random_state	42
cv_folds	5
eval_ratio	0.2
n_features	190
train_samples	246008
eval_samples	61503
target_rate	0.0807

3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

STEP 4. train_all:
boucle catalogue + MLflow logging

STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

STEP 7. register_db:
PostgreSQL model_versions

3. Enregistrer Expérimentations dans MLFlow

✅ **Enregistrement des modèles dans la base de données PostgresSQL**

id [PK] integer	
model_name character varying (255)	
version character varying (50)	
mlflow_run_id character varying (255)	
algorithm character varying (100)	
hyperparameters jsonb	
metrics jsonb	
1	
	1
logistic_regression	
v_20260223_131952	
a955f7b12ba741628edf6361e61765b8	
LogisticRegression	
{"C": "0.1", "tol": "0.0001", "dual": "False", "n_jobs": "-1", "solver": "lbfgs", "penalty": "deprecated", "verbose": "0", "l1_ratio": "0.0", "max_iter": "1000", "warm_start": "False", "class_weight":...	
{"f1": 0.3181818181818182, "f2": 0.5147058823529411, "mcc": 0.31638586424905013, "recall": 0.875, "roc_auc": 0.782608695652174, "accuracy": 0.7, "log_loss": 0.5967232287148224, "precision": 0.19444444...	
2	
	2
decision_tree	
v_20260223_151416	
e045c4255a7f4314990cdf296e3a3f81	
DecisionTreeClassifier	
{"splitter": "best", "ccp_alpha": "0.0", "criterion": "gini", "max_depth": "8", "class_weight": "balanced", "max_features": "None", "random_state": "42", "monotonic_cst": "None", "max_leaf_nodes": "No...	
{"f1": 0.3, "f2": 0.5172413793103449, "mcc": 0.3520012515622306, "recall": 1.0, "roc_auc": 0.8439716312056738, "accuracy": 0.72, "log_loss": 0.4762024823115933, "precision": 0.17647058823529413, "cohe...	
3	

3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

STEP 4. train_all:
boucle catalogue + MLflow logging

STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

STEP 7. register_db:
PostgreSQL model_versions

3. Enregistrer Expérimentations dans MLFlow

✅ **Comparé les performances entre modèles de manière rigoureuse.**

Échantillons Train..: 8000
Échantillons Eval...: 2000

```
================================================================================================================
COMPARAISON DES MODÈLES — Phase 3   (trié par F2-Score eval)
================================================================================================================
Rang              Modèle  AUC-ROC(eval)  F2(eval)  F1(eval)  Recall(eval)  Precision(eval)  MCC(eval)  F1 CV mean  F1 CV std  Overfit ΔF1  Train (s) Overfitting MLflow Run
   🏆        decision_tree         0.6962    0.3875    0.2523        0.6026           0.1596     0.1966      0.2264     0.0198        0.0317       0.42                ✅ 49006d6c
      logistic_regression         0.7355    0.3750    0.2365        0.6154           0.1463     0.1780      0.2604     0.0113        0.0455       1.59                ✅ b346577a
                  xgboost         0.7282    0.3352    0.2431        0.4487           0.1667     0.1705      0.2828     0.0280        0.2532       3.33                ❗ 89128889
                 lightgbm         0.7271    0.3340    0.2612        0.4103           0.1916     0.1897      0.2746     0.0224        0.3344      19.76                ❗ ea06a1f0
            random_forest         0.6944    0.2768    0.2534        0.2949           0.2222     0.1827      0.2551     0.0251        0.3984       3.34                ❗ e21fb06d
                      mlp         0.6395    0.1501    0.1544        0.1474           0.1620     0.0866      0.1546     0.0572        0.7509      25.60                ❗ 2831076f
                    [illegible]   0.5072    0.0761    0.0750        0.0769           0.0732    -0.0054      0.0814     0.0135        0.0007       0.00                ✅ [illegible]
                    [illegible]   0.7412    0.0079    0.0121        0.0064           0.1111     0.0083      0.0459     0.0325        0.2041      28.31                ❗ 16284d41
================================================================================================================
Rang  Modèle  AUC-ROC(eval)  F2(eval)  F1(eval)  Recall(eval)  Precision(eval)  MCC(eval)  F1 CV mean  F1 CV std  Overfit ΔF1  Train (s) Overfitting MLflow Run
   ✅ 🏆 Champion : decision_tree  (F2=0.3875  AUC-ROC=0.6962)
   ☐ 🏆  Phase 4 : Optuna pour optimiser les hyperparamètres du champion.
                 lightgbm         0.7515    0.4166    0.2666        0.6667           0.1666     0.2177      0.2678     0.0050        0.0171      10.78                ✅ 9bc1cced
                  xgboost         0.7495    0.4139    0.2622        0.6737           0.1628     0.2133      0.2633     0.0048        0.0096      11.96                ✅ e4b7e7ef
      logistic_regression         0.7341    0.3964    0.2499        0.6504           0.1547     0.1946      0.2567     0.0032        0.0086      34.84                ✅ c3db6fed
            random_forest         0.7272    0.3895    0.2566        0.5952           0.1635     0.1961      0.2608     0.0039        0.0432      42.95                ✅ 719e3da8
                [illegible]   0.7028    0.3777    0.2218        0.7104           0.1314     0.1638      0.2273     0.0050        0.0042       6.30                ✅ [illegible]
                  dummy_baseline  0.5016    0.0830    0.0832        0.0828           0.0837     0.0032      0.0822     0.0054       -0.0025       0.02                ✅ [illegible]
                      mlp         0.6999    0.0741    0.1013        0.0628           0.2609     0.0931      0.1165     0.0088        0.0548     193.60                ✅ e07bfce3
   Rang  gradient_boosting  AUC-ROC(eval)  F2(eval)  F1(eval)  Recall(eval)  Precision(eval)  MCC(eval)  F1 CV mean  F1 CV std  Overfit ΔF1  Score Moyen  Seuil Moyen  TN opt  Train (s) Overfitting MLflow Run
   🏆                 lightgbm         0.7664    0.4296    0.2773        0.6777           0.1743     0.2323      0.2785     0.0034        0.0212        31767        0.527    1780       13967      17.[illegible] ✅ 9efa66ce 5d615923
                      xgboost         0.7638    0.4245    0.2705        0.6842           0.1686     0.2251      0.2731     0.0043        0.0145        31930        0.535    1787       14060      17.[illegible] ✅ ca1c9150
          logistic_regression         0.7484    0.4129    0.2604        0.6773           0.1612     0.2116      0.2627     0.0045        0.0038        33180        0.550    1947       13710      108.[illegible] ✅ 564b6f82
                random_forest         0.7396    0.3985    0.2676        0.5915           0.1729     0.2090      0.2729     0.0053        0.0606        33898        0.482    1754       16358      58.[illegible] ✅ 31ccbaa6
                dummy_baseline  0.5016    0.0830    0.0832        0.0828           0.0837     0.0032      0.0822     0.0054       -0.0025        50041        0.050    4554       4501       0.[illegible] ✅ 6154fd03
                      mlp         0.7151    0.0699    0.0976        0.0588           0.2874     0.0983      0.1658     0.0074        0.0840        35620        0.073    1787       17750      372.[illegible] ✅ 3bb9051a
            gradient_boosting         0.7550    0.0098    0.0155        0.0079           0.5270     0.0569      0.0189     0.0019        0.0061        32669        0.088    1795       14719      868.[illegible] ✅ 38a93b97
```

Échantillons Train..: 246008
Échantillons Eval...: 61503

Échantillons Train..: 246008
Échantillons Eval...: 61503

3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

STEP 4. train_all:
boucle catalogue + MLflow logging

STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

STEP 7. register_db:
PostgreSQL model_versions

3. Enregistrer Expérimentations dans MLFlow

✅ Comparé les performances : Phase 3 Training

Modèle	AUC-ROC	F2-Score	Score Métier	Seuil Mét.	FN (opt)	FP (opt)	Déséquilibre	Train (s)	Statut
LightGBM 🏆	0.766	**0.430**	31,767	0.527	1,780	13,967	`class_weight`	17.7	✅ BEST
XGBoost	0.764	**0.425**	31,930	0.535	1,787	14,060	`scale_pos_weight`	17.5	✅
Log. Regression	0.748	**0.413**	33,180	0.550	1,947	13,710	`class_weight`	108.7	✅
Random Forest	0.740	**0.399**	33,898	0.482	1,754	16,358	`class_weight`	58.6	✅
Decision Tree	0.703	**0.378**	36,432	0.527	2,217	14,262	`class_weight`	14.4	✅
MLP (Neural Net)	0.715	**0.070**	35,620	0.073	1,787	17,750	`SMOTE`	372.4	⚠️ Exclure
Grad. Boosting	0.755	**0.010**	32,669	0.088	1,795	14,719	`aucun`	869.0	⚠️ Exclure

1. Suprématie des GBDT (Gradient Boosted Decision Trees)

Les modèles basés sur le boosting d'arbres dominent largement le classement. LightGBM s'impose comme le champion incontesté avec un F2-Score de 0,430 et un AUC-ROC de 0,766. Il offre le meilleur compromis entre la capacité de discrimination (AUC) et la détection des risques réels ($F2$).

2. Analyse de la Performance Métier

- L'observation du Score Métier et des Faux Négatifs (FN) révèle l'impact financier :
- Priorité au Rappel : Bien que les Faux Positifs (FP) soient nombreux (~14 000), le modèle minimise les pertes lourdes en identifiant avec précision les clients à risque (FN de 1 780 pour LightGBM).
- Seuil d'Optimisation : Le seuil métier optimal se stabilise autour de 0,53. Ce réglage est crucial : il définit la limite de tolérance au risque pour l'institution financière.

3. Stratégies face au Déséquilibre (Class Imbalance)

- Le traitement du déséquilibre des classes a été le facteur déterminant de succès :
- Poids des classes (class_weight) : L'utilisation de poids asymétriques a permis aux modèles linéaires et aux arbres de surpasser les approches par échantillonnage.
- Échec du SMOTE sur MLP : L'approche par données synthétiques (SMOTE) n'a pas été concluante pour le réseau de neurones, générant un F2-Score très faible (0,070), soulignant la complexité des données tabulaires.

4. Efficacité Opérationnelle

- En termes de déploiement (MLOps), LightGBM et XGBoost sont les plus efficients :
- Rapidité : Un temps d'entraînement de seulement 17,7 secondes, contre plus de 6 minutes pour le MLP.
- Robustesse : Une grande stabilité des résultats malgré la forte dimensionnalité des données issues de la Phase 2.

La prochaine étape de la **Phase 4 (Optuna)** consistera à tenter de réduire ce **Score Métier** en dessous de la barre des **31 000** grâce à l'optimisation des hyperparamètres. »

3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

STEP 4. train_all:
boucle catalogue + MLflow logging

STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

STEP 7. register_db:
PostgreSQL model_versions

3. Enregistrer Expérimentations dans MLFlow

✅ **Feature importance après entraînement d'un modèle.**

lightgbm_feature_importance.csv

	feature	importance
1	standard__ext_source_3	486
2	standard__ext_source_2	472
3	standard__ext_source_1	408
4	standard__days_birth	399
5	log__amt_credit	330
6	log__amt_annuity	268
7	standard__days_employed	219
8	log__amt_goods_price	217
9	standard__days_id_publish	197
10	standard__days_last_phone_change	169
11	log__amt_income_total	164
12	standard__days_registration	158
13	standard__region_population_relative	102
14	ohe__name_contract_type_cash_loans	91
15	standard__own_car_age	86
16	standard__hour_appr_process_start	86
17	ordinal__occupation_type	73
18	log__amt_req_credit_bureau_year	71
19	ohe__code_gender_f	59
20	ohe__name_education_type_higher_education	56

gradient_boosting_feature_importance.csv

	feature	importance
1	standard__ext_source_3	0.33822171945922325
2	standard__ext_source_2	0.3167944436896558
3	standard__ext_source_1	0.08172701083164986
4	standard__days_birth	0.039156059680830205
5	ohe__code_gender_m	0.02593843900898112
6	log__amt_goods_price	0.02434908904815332
7	log__amt_credit	0.021898590403012723
8	standard__days_employed	0.019437898219492857
9	log__amt_annuity	0.018135483957282452
10	ohe__name_education_type_higher_education	0.01656199221297225
11	ohe__name_income_type_working	0.010856493237527451
12	standard__days_id_publish	0.006808314312912166
13	standard__own_car_age	0.006276296054091234
14	ohe__name_education_type_secondary_secondary_special	0.005833628158042734
15	standard__missingindicator_own_car_age	0.005547470995541985
16	standard__missingindicator_ext_source_3	0.00533759549857953
17	standard__days_last_phone_change	0.005283744437912444
18	ohe__name_family_status_married	0.005156048020001223
19	standard__region_rating_client_w_city	0.004891565822004579
20	ohe__name_contract_type_cash_loans	0.0036079984678670097

3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

STEP 4. train_all:
boucle catalogue + MLflow logging

STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

STEP 7. register_db:
PostgreSQL model_versions

3. Enregistrer Expérimentations dans MLFlow

✅ **Optimise le seuil de décision en fonction du coût métier.**

```
======================================================

🚀 PHASE 3 — CLASSIFICATION + MLFLOW TRACKING
   Projet : Prêt à Dépenser — Home Credit Default Risk
======================================================

📊 Configuration MLflow ...
ℹ️  Expérience existante (ID=2)
✅ MLflow prêt → http://localhost:5001

======================================================

📊 Chargement données (source=db) ...
ℹ️  Lecture des tables ml_X_train et ml_y_train ...
✅ Tables SQL chargées en 6.5s — 10,000 lignes × 138 colonnes
ℹ️  Remboursés (0) :   9,222  (92.2%)
ℹ️  Défauts    (1) :     778  (7.8%)
⚠️  Déséquilibre   : 1 défaut pour ~11 remboursements
→ class_weight='balanced' / scale_pos_weight=11

======================================================

📊 Split stratifié — train 80% / eval 20% ...
⚠️  Colonnes string ignorées : ['target_enc__organization_type']
✅ X_train : (8000, 136)  défauts=7.8%
✅ X_eval  : (2000, 136)  défauts=7.8%
ℹ️  Features totales : 136
```

```
================================================================================
✅ 🏆 Champion : decision_tree  (F2=0.3875  AUC-ROC=0.6962)
ℹ️  □  Phase 4 : Optuna pour optimiser les hyperparamètres du champion.

================================================================================
📊 Sauvegarde des modèles ...
ℹ️     ✓ decision_tree                  → decision_tree_model.joblib
✅ 1 modèle(s) dans C:\Users\Public\IAE_DELL\pra_dell\m6_ocr\models/

================================================================================
📊 Optimisation du seuil pour : decision_tree
✅ Seuil optimal trouvé..: 0.482
ℹ️  Nouveau F2-Score.....: 0.3875

================================================================================
📊 Enregistrement champion [decision_tree] dans PostgreSQL ...
✅ ✅ Champion enregistré en DB (Version: v_20260225_153745)

================================================================================
✅ PHASE 3 TERMINÉE AVEC SUCCÈS
================================================================================
  Durée totale........: 0:15:05 (H:M:S)
  Modèles entraînés...: 8
  Champion............: decision_tree
  MLflow UI...........: http://localhost:5001
  Expérience..........: Smart_Credit_Scoring

  □  Suite : Phase 4 — Optimisation hyperparamètres
================================================================================
```

3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

STEP 4. train_all:
boucle catalogue + MLflow logging

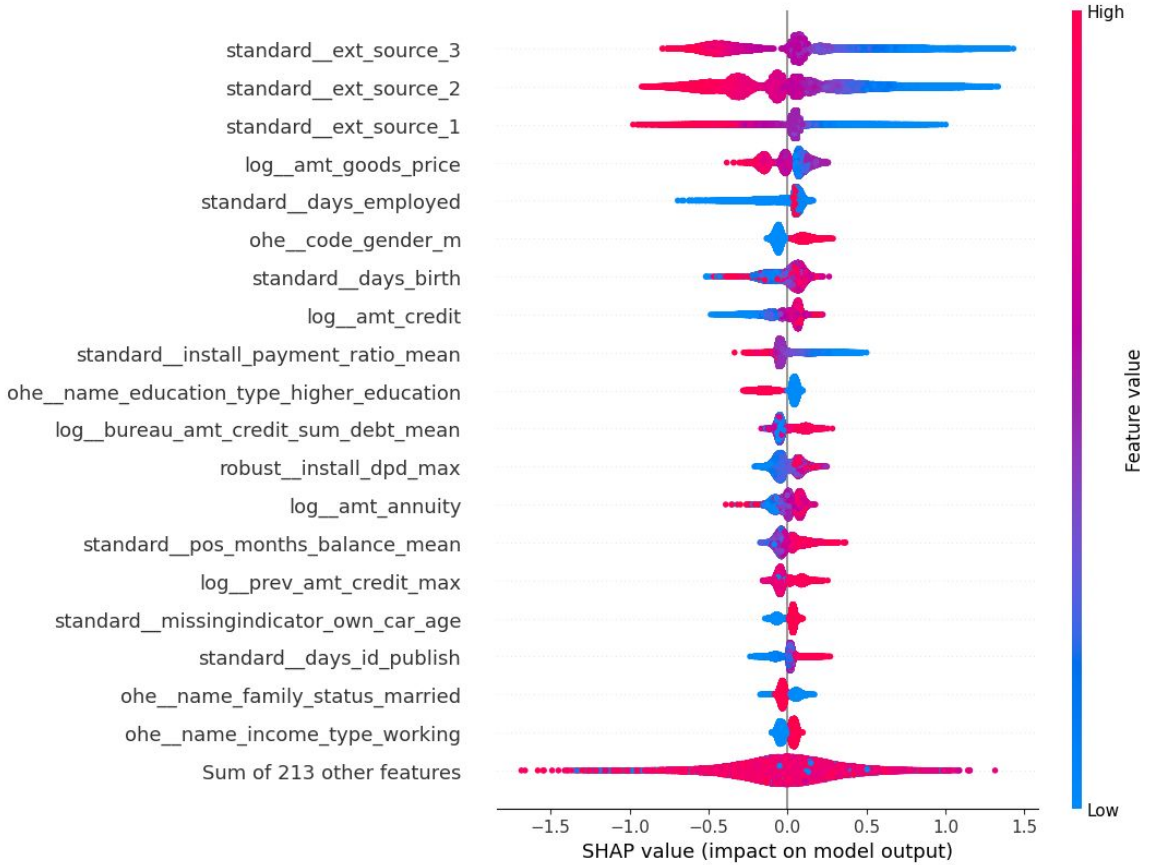
STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

STEP 7. register_db:
PostgreSQL model_versions

3. Enregistrer Expérimentations dans MLFlow

📊 Beeswarm Plot — Top 20 variables (impact sur la probabilité de défaut)



3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

STEP 4. train_all:
boucle catalogue + MLflow logging

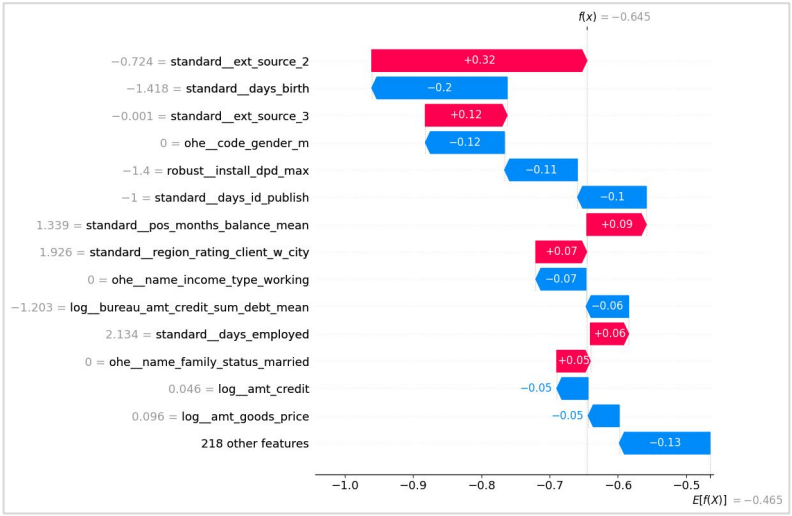
STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

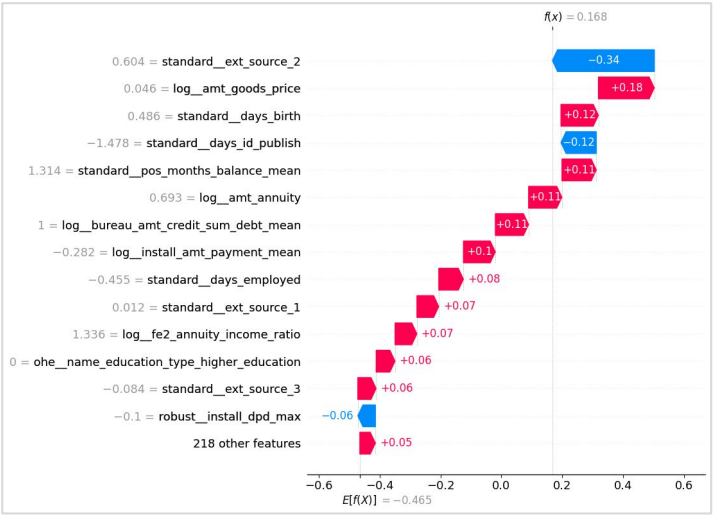
STEP 7. register_db:
PostgreSQL model_versions

3. Enregistrer Expérimentations dans MLFlow

📋 Analyse — BON CLIENT (TARGET=0)



📋 Analyse — CLIENT EN DÉFAUT (TARGET=1)



3

Enregistrer Expérimentations dans MLFlow

STEP 0. setup_mlflow:
configure tracking + experience

STEP 1. load_data:
PostgreSQL v_features_engineering

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. init_modeler:
ClassificationModeler

STEP 4. train_all:
boucle catalogue + MLflow logging

STEP 5. compare:
tableau comparatif + champion

STEP 6. save:
joblib + metadata JSON

STEP 7. register_db:
PostgreSQL model_versions

3. Enregistrer Expérimentations dans MLFlow

RAPPORT DE SYNTHÈSE — MODÈLE DE SCORING CRÉDIT

```
======================================================================
  RAPPORT DE SYNTHÈSE — MODÈLE DE SCORING CRÉDIT
======================================================================
  Modèle champion     : LIGHTGBM
  Run MLflow          : b7a76deb...
  AUC-ROC (eval)      : 0.7663815515428188
  F2-Score (eval)     : 0.42959274862760116

  Seuil optimal       : 0.528  (vs 0.50 par défaut)
  Score Métier opt.   : 31788  (10×1788 + 1×13908)

  TOP 5 VARIABLES LES PLUS INFLUENTES (SHAP)
  ------------------------------------------------
    1. standard__ext_source_3                  SHAP=0.3309
    2. standard__ext_source_2                  SHAP=0.3274
    3. standard__ext_source_1                  SHAP=0.1457
    4. log__amt_goods_price                    SHAP=0.0987
    5. standard__days_employed                 SHAP=0.0848
```

```
ARTEFACTS PRODUITS
--------------------------------------------------
  ✅ .gitkeep
  ✅ .ipynb_checkpoints
  ✅ confusion_matrices.png
  ✅ feature_importance_global.png
  ✅ figures
  ✅ phase4_comparison_p3_p4_decision_tree.png
  ✅ phase4_gs_cv_results_decision_tree.csv
  ✅ phase4_gs_heatmaps_decision_tree.png
  ✅ phase4_gs_top_combos_decision_tree.png
  ✅ phase4_optuna_history_mpl_decision_tree.png
  ✅ phase4_optuna_study_decision_tree.pkl
  ✅ phase4_threshold_analysis_decision_tree.png
  ✅ phase4_threshold_curve.csv
  ✅ roc_pr_curves.png
  ✅ shap_importance_globale.csv
  ✅ threshold_curve.png
```

INTERPRÉTATION MÉTIER

- Un score élevé sur EXT_SOURCE_* = client externe bien noté → ↓ risque
- DAYS_EMPLOYED faible (emploi récent) → ↑ risque de défaut
- AMT_CREDIT élevé par rapport aux revenus → ↑ risque

4

Optimisation hyperparamètres

STEP 0. setup_mlflow:
configure + récupérer expérience

STEP 1. load_champion:
Charge modèle champion (joblib + metadata JSON)

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. GridSearchCV:
StratifiedKFold 5-plis + scorer F2

STEP 4. Optuna:
Optimisation bayésienne

STEP 5. compare_engines:
(GridSearch vs Optuna) selon F2 eval

STEP 6. optimize_threshold:
seuil de décision optimal selon coût métier

STEP 7. save:
modèle optimisé et métadonnées

STEP 8. register_db:
Insert champion Phase 4

STEP 9. register_monitoring:
Sauvegarde en DB métriques de référence

4. Optimisation hyperparamètres

✅ **Optimisation d'hyperparamètres (GridSearchCV / Optuna).**

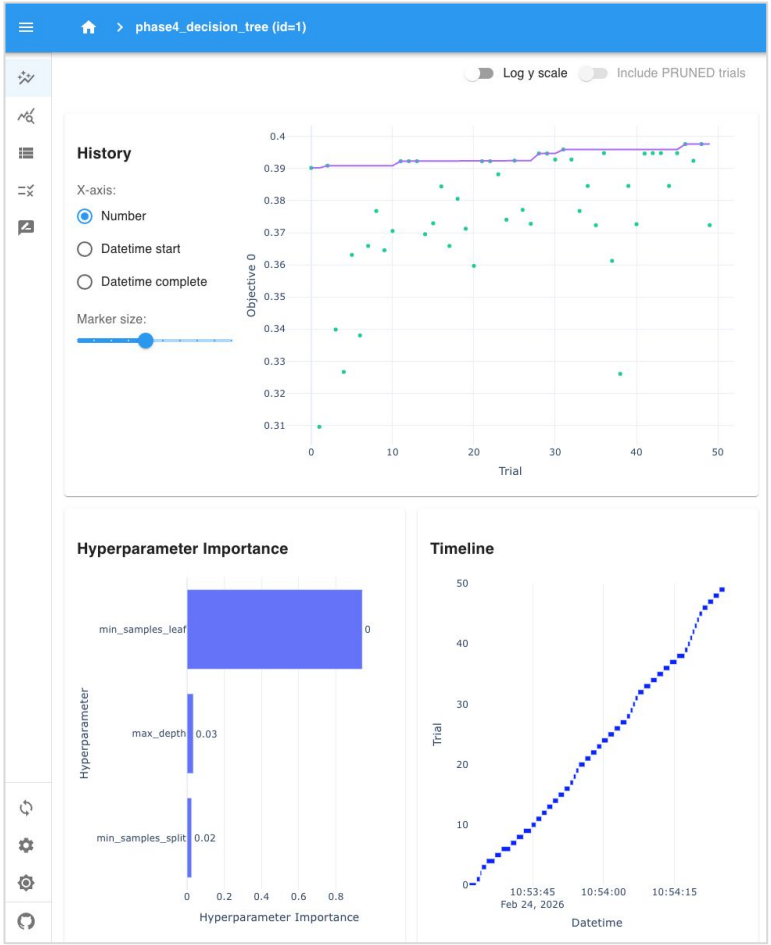
✅ **Défini une fonction de coût métier prenant en compte FN > FP.**

```
COUT_FN = 10  # coût d'un Faux Négatif (défaut non détecté)
COUT_FP = 1   # coût d'un Faux Positif (refus injustifié)


def optimize_threshold(
    y_true:     np.ndarray,
    y_proba:    np.ndarray,
    cout_fn:    int   = COUT_FN,
    cout_fp:    int   = COUT_FP,
    n_steps:    int   = 100,
    beta:       float = 2.0,
) -> Tuple[float, pd.DataFrame]:
    """
    Détermine le seuil de décision optimal selon deux critères :
        1. Coût métier total minimisé  (FN × cout_fn + FP × cout_fp)
        2. F2-Score maximisé           (beta=2 → recall prioritaire)
```

✅ **Testé plusieurs seuils de décision pour classifier les clients.**

```
thresholds = np.linspace(0.05, 0.80, 100)
for t in thresholds:
    y_pred = (y_probs >= t).astype(int)
    # Calcul du coût métier associé à ce seuil...
```



4

Optimisation hyperparamètres

STEP 0. setup_mlflow:
configure + récupérer expérience

STEP 1. load_champion:
Charge modèle champion (joblib + metadata JSON)

STEP 2. split:
stratifié 80% train / 20% eval

STEP 3. GridSearchCV:
StratifiedKFold 5-plis + scorer F2

STEP 4. Optuna:
Optimisation bayésienne

STEP 5. compare_engines:
(GridSearch vs Optuna) selon F2 eval

STEP 6. optimize_threshold:
seuil de décision optimal selon coût métier

STEP 7. save:
modèle optimisé et métadonnées

STEP 8. register_db:
Insert champion Phase 4

STEP 9. register_monitoring:
Sauvegarde en DB métriques de référence

4. Optimisation hyperparamètres

✅ Résumé de résultats: Validation Croisée (CV) vs. Évaluation Holdout

- **Le "Holdout" (Jeu de Test) :** C'est le groupe de **61 503 clients** totalement isolés pendant l'entraînement. Il simule la réalité du déploiement en production sur des données inconnues.
- **Les "Folds" (Validation Croisée) :** Ce sont des sous-groupes de **~49 000 clients** utilisés pour des "simulations d'entraînement". Le score CV est la moyenne de ces simulations, garantissant la stabilité du modèle.

	GridSearch	Optuna	Mejora
CV coût	28,822	28,670	✅ -152
Eval F2	0.3854	0.3866	✅ +0.0012
Eval AUC	0.7116	0.7143	✅ +0.0027
Recall	0.6645	0.6649	✅ +0.0004
Precision	0.1438	0.1446	✅ +0.0008
Coût métier	36,306	36,173	✅ -133
FN	1,666	1,664	✅ -2
FP	19,646	19,533	✅ -113
Seuil optimal	0.500	0.538	✅ -359 coût
Tiempo	747s	3,640s	—

Interprétation Stratégique

"Le **Holdout** est notre 'bac à sable' de réalité : il nous donne la facture réelle que le modèle générerait pour la banque. Bien que le coût CV paraisse plus bas (~28k), c'est uniquement parce qu'il porte sur des échantillons plus petits (~49k).

L'important ici est la baisse du **Coût Métier** sur le Holdout grâce à **Optuna**. En passant de 36 306 à 36 173, l'optimisation bayésienne a mieux généralisé. Nous avons sauvé 2 dossiers de défauts critiques (très coûteux) et évité de rejeter 113 bons clients par rapport à une recherche classique par grille.

Q & A

Rafael CEREZO MARTIN Février 2026